Guía rápida de Pandas

Esta guía recoge **solo lo que usa la práctica**. No hace falta aprenderla de memoria: tenla a mano mientras completas el cuaderno. Algunos ejemplos usan la tabla `hoteles` de la práctica; los de leer y guardar usan archivos inventados, que tendrás que adaptar a cada fuente.

1. Qué es Pandas

Pandas es una biblioteca de Python para trabajar con **datos en forma de tabla**: leerlos de muchos formatos (CSV, Excel, JSON, bases de datos…), mirarlos, transformarlos y guardarlos. Es una de las herramientas más usadas en ciencia de datos e ingeniería de datos. Colab ya la trae instalada.

Al principio de cada cuaderno se importa con un nombre corto, **pd**, que es la costumbre en todo el mundo:

```
import pandas as pd      # a partir de aquí, «pd» significa «pandas»
pd.__version__           # qué versión tengo instalada
```

2. Las estructuras de datos de Pandas

Pandas organiza los datos en **tres estructuras** que encajan unas dentro de otras:

Estructura	Qué es	Ejemplo en la práctica
DataFrame	Una **tabla** completa: filas y columnas. Es con lo que trabajarás casi siempre.	`hoteles`, `viviendas`, `ocupacion`
Series	**Una sola columna** (o una sola fila) de un DataFrame: una lista de valores con etiquetas.	`hoteles["plazas"]`
Index	Las **etiquetas** de las filas (normalmente 0, 1, 2…) o los nombres de las columnas.	`hoteles.index`, `hoteles.columns`

Index de columnas: hoteles.columns

	establecimiento_id	direccion_isla_nombre	plazas
0	-38-4-0000001	Tenerife	0
1	-38-4-0000003	Tenerife	0
2	A-38-4-0006100	Tenerife	270
…	…	…	…
847	…	…	…

■ Index: etiquetas de filas y de columnas

■ Series: una columna, por ejemplo la de islas

Todo junto: el DataFrame `hoteles`, de 848 filas × 15 columnas

Para sacar una columna de un DataFrame se escribe su nombre **exacto** entre corchetes y comillas. El resultado es una Series:

```
islas = hoteles["direccion_isla_nombre"]     # una Series con 848 valores
type(islas)                                  # pandas.core.series.Series
```

3. Tipos de datos de cada columna (dtype)

Cada columna de un DataFrame tiene un **tipo de dato**, el **dtype**: todos sus valores son del mismo tipo. Se consulta con `df.dtypes` o con `df.info()`.

dtype	Qué guarda	Ejemplo
`int64`	Números enteros	Número de páginas de un libro: 328
`float64`	Números con decimales	Precio: 12.95
`bool`	Verdadero o falso (True / False)	¿Está prestado? True
`object` o `str`	Texto	Títulos, nombres, códigos
`datetime64`	Fechas y horas	2026-10-21 20:58:17
`category`	Texto que se repite mucho con pocos valores distintos	Género: novela, ensayo, poesía

`NaN` (*Not a Number*) es como Pandas marca un **valor que falta**, por ejemplo una celda vacía.

> **¿`object` o `str`?** Son lo mismo para esta práctica: es el tipo «texto». Colab muestra `object`; las versiones más nuevas de Pandas (3.0) muestran `str`.

Quién decide el dtype al leer

Al leer un archivo, Pandas **intenta adivinar** el tipo de cada columna mirando su contenido. Con el parámetro `dtype=str` no adivina nada: lee **todas las columnas como texto**. Por ejemplo, con un catálogo de libros:

```
# libros.csv
# isbn;titulo;paginas
# 0451524934;1984;328

pd.read_csv("libros.csv", sep=";").dtypes              # Pandas adivina
# isbn       int64
# titulo    object
# paginas    int64

pd.read_csv("libros.csv", sep=";", dtype=str).dtypes   # todo como texto
# isbn      object
# titulo    object
# paginas   object
```

Del mismo modo, Pandas convierte en `NaN` las celdas vacías y algunos textos como «NA» o «null». Con `keep_default_na=False` no lo hace: las celdas vacías se quedan como texto vacío. Al hacer la práctica, piensa qué opción conviene cuando el objetivo es guardar los datos **tal como vienen**.

4. Leer datos: las funciones pd.read_…

Para cada formato hay una función que devuelve un **DataFrame**. Los ejemplos usan archivos inventados: en el cuaderno tendrás que adaptarlos a cada fuente.

Función	Formato	Parámetros más habituales	Ejemplo
`pd.read_csv()`	CSV	`sep` separador de columnas · `encoding` codificación de caracteres · `dtype` tipos · `keep_default_na` celdas vacías	`pd.read_csv("libros.csv", sep=";")`
`pd.read_excel()`	Excel	`sheet_name` hoja, por número (0 = la primera) o por nombre · `dtype`	`pd.read_excel("notas.xlsx", sheet_name="Trimestre 1")`
`pd.json_normalize()`	JSON ya cargado en Python	Recibe una **lista de registros** y la convierte en tabla	`pd.json_normalize(datos["alumnos"])`
`pd.read_sql()`	Base de datos	Una consulta SQL y una conexión · `params` valores para los ? de la consulta	`pd.read_sql("SELECT * FROM libros", conexion)`

Un JSON que llega por internet

Un JSON se pide al servidor con la biblioteca **requests**, que no es de Pandas. La respuesta se convierte en un **diccionario de Python**, y la lista de registros que hay dentro se pasa a Pandas. Por ejemplo, si un servidor devolviera `{"alumnos": [{"nombre": "Ana", "curso": 1}, …]}`:

```
respuesta = requests.get("https://ejemplo.org/alumnos.json")   # 1. pedir el archivo
datos = respuesta.json()                    # 2. texto JSON  →  diccionario de Python
datos.keys()                                # 3. mirar qué claves tiene: dict_keys(['alumnos'])
alumnos = pd.json_normalize(datos["alumnos"])   # 4. lista de registros  →  DataFrame
```

El paso 3 es importante: antes de convertir, mira la estructura del JSON para saber en qué clave está la lista.

5. Mirar los datos sin modificarlos

Antes de guardar, siempre hay que **mirar** lo que se ha leído. Ninguna de estas instrucciones cambia el DataFrame:

Instrucción	Qué muestra	En la práctica
`df.shape`	(filas, columnas)	`hoteles.shape` → (848, 15)
`len(df)`	Número de filas	`len(viviendas)` → 72176
`df.head()`	Las 5 primeras filas (`df.head(10)`: las 10 primeras)	Ver cómo vienen los datos
`df.tail()`	Las 5 últimas filas	Comprobar que el final no está cortado
`df.info()`	Columnas, valores no vacíos y dtype de cada una	Detectar columnas con huecos
`df.columns`	Los nombres de las columnas	Copiar un nombre exacto
`df.dtypes`	El dtype de cada columna	Ver qué ha adivinado Pandas
`serie.value_counts()`	Cuántas veces aparece cada valor de una Series (una columna)	`islas.value_counts()` → Tenerife 316, Gran Canaria 236…

¿Con paréntesis o sin ellos? Las **funciones y métodos** hacen algo y llevan paréntesis: `df.head()`, `df.info()`. Los **atributos** son datos que el DataFrame ya tiene y van sin paréntesis: `df.shape`, `df.columns`, `df.dtypes`. Si escribes `df.shape()` obtendrás un error: `TypeError: tuple object is not callable`.

6. Guardar datos: los métodos df.to_…

Para guardar, el DataFrame tiene un método por formato. Se le indica la ruta del archivo:

```
libros.to_csv("libros.csv", index=False)          # guarda en CSV
libros.to_parquet("libros.parquet", index=False)  # guarda en Parquet
```

¿Qué hace `index=False`? Sin él, Pandas escribe también el índice de filas (0, 1, 2…) como una columna más del archivo. Ese número no viene de la fuente: lo pone Pandas al leer.

Características de cada formato

Con estas características, y con las medidas que hagas en el cuaderno, podrás justificar qué formato eliges:

Formato	Tipo de archivo	Cómo organiza los datos	¿Guarda el dtype de cada columna?	¿Lo lee una persona?
CSV	Texto	Por filas, separadas por un carácter	No	Sí, con cualquier editor
Parquet	Binario, comprimido	Por columnas	Sí	No, hace falta un programa
JSON	Texto	Registros con claves y valores; admite datos anidados	En parte (texto, números, verdadero/falso)	Sí, con cualquier editor
Excel	Binario	Hojas con celdas	En parte (cada celda tiene su tipo)	Sí, con una hoja de cálculo

Para guardar un JSON tal como llegó se usa `json.dump()`, una función de Python, no de Pandas.

7. Errores frecuentes con Pandas

Mensaje	Causa habitual	Solución
`NameError: name 'pd' is not defined`	No se ha ejecutado `import pandas as pd`	Ejecuta la celda de configuración (apartado 1)
`KeyError: 'Plazas'`	El nombre de la columna no es exacto (mayúsculas, tildes, espacios)	Copia el nombre de `df.columns`
`TypeError: … object is not callable`	Paréntesis en un atributo: `df.shape()`	Quita los paréntesis: `df.shape`
`AttributeError: … has no attribute 'head'`	La variable no es un DataFrame (por ejemplo, la extracción ha fallado o falta completar un `...`)	Comprueba con `type(variable)` qué contiene
`FileNotFoundError`	La ruta del archivo no es correcta	Revisa la carpeta y el nombre exacto del archivo

8. Para saber más

La documentación oficial de Pandas está en inglés y es muy clara. Para empezar:

• **10 minutes to pandas**, un recorrido rápido: pandas.pydata.org/docs/user_guide/10min.html

• **Getting started tutorials**, diez tutoriales cortos con dibujos. Para esta práctica, los dos primeros: *What kind of data does pandas handle?* y *How do I read and write tabular data?*: pandas.pydata.org/docs/getting_started/intro_tutorials

• En Colab, escribe una función seguida de **?** y ejecuta la celda para ver su ayuda: `pd.read_csv?`